

LeetCode Java Array Problem-Solving Guide

15 Problems • Arrays • Pointers • Logic

A Beginner-Friendly Problem-Solving Workbook

Table of Contents

#	Problem Name	Pattern / Topic	Difficulty
1480	Running Sum of 1d Array	Prefix Sum	Easy
1920	Build Array from Permutation	Array Translation	Easy
1929	Concatenation of Array	Array Copy	Easy
1470	Shuffle the Array	Two Pointers	Easy
1431	Kids With the Greatest Number of Candies	Array Max	Easy
1672	Richest Customer Wealth	2D Array	Easy
1512	Number of Good Pairs	Frequency Array	Easy
1365	How Many Numbers Are Smaller	Nested Loops	Easy
1295	Find Numbers with Even Number of Digits	Digit Extraction	Easy
485	Max Consecutive Ones	Counting State	Easy
66	Plus One	Array Math	Easy
27	Remove Element	Two Pointers (R/W)	Easy
26	Remove Duplicates from Sorted Array	Two Pointers (R/W)	Easy
283	Move Zeroes	Two Pointers (Swap)	Easy
905	Sort Array By Parity	Two Pointers (Swap)	Easy

Learning Focus: Arrays are the foundation of data structures. This workbook emphasizes understanding how indices work, how to maintain state across loops, and the extremely powerful "Two Pointer" technique.

Problem 1480 — Running Sum of 1d Array

EASY

PATTERN: PREFIX SUM

1. UNDERSTAND THE PROBLEM

Given: An integer array `nums`.

Find: A running sum array where $\text{runningSum}[i] = \text{sum}(\text{nums}[0] \dots \text{nums}[i])$.

Example: Input: `nums = [1, 2, 3, 4]`

Output: `[1, 3, 6, 10]`

Explanation: `[1, 1+2, 1+2+3, 1+2+3+4]`

2. KEY IDEA

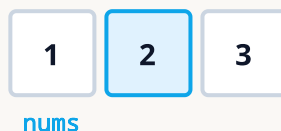
LOOK BACK ONE STEP

Instead of recalculating the sum from the beginning every time, just add the current number to the *previous* running sum.

$\text{current_sum} = \text{previous_sum} + \text{current_number}$

3. VISUALIZE THE ALGORITHM

Step 1: Copy the first element directly.



Step 2: $\text{ans}[1] = \text{ans}[0] + \text{nums}[1] (1 + 2 = 3)$



1. Create a new array `ans` of the same length as `nums`.
2. Set the first element: $\text{ans}[0] = \text{nums}[0]$.
3. Loop from index 1 to the end.
4. For each index `i`, set $\text{ans}[i] = \text{ans}[i-1] + \text{nums}[i]$.
5. Return `ans`.

4. JAVA SOLUTION

```
class Solution {
    public int[] runningSum(int[] nums) {
        int[] ans = new int[nums.length];

        // The first element is always the same
        ans[0] = nums[0];

        // Start loop from index 1
        for (int i = 1; i < nums.length; i++) {
            // Current running sum = previous running sum + current number
            ans[i] = ans[i - 1] + nums[i];
        }

        return ans;
    }
}
```

5. DRY RUN

Input: nums = [1, 2, 3, 4]

Initialization	i	nums[i]	ans[i-1]	ans[i] (Calculation)	ans Array State
ans[0] = 1	-	-	-	-	[1, 0, 0, 0]
Loop starts	1	2	1	1 + 2 = 3	[1, 3, 0, 0]
Continue	2	3	3	3 + 3 = 6	[1, 3, 6, 0]
Continue	3	4	6	6 + 4 = 10	[1, 3, 6, 10]

6. COMPLEXITY

Time: O(N)

Space: O(N)

Time: We visit each element in the array exactly once, doing a single addition operation. N is the length of the array.

Space: We create a new array ans of size N to hold the result. (Note: If allowed to modify the input array, this can be done in O(1) auxiliary space, but creating a new array is generally safer practice).

Problem 1920 — Build Array from Permutation

EASY

PATTERN: ARRAY TRANSLATION

1. UNDERSTAND THE PROBLEM

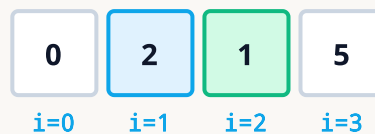
Given: A zero-based permutation array `nums`.

Find: An array `ans` of the same length where `ans[i] = nums[nums[i]]`.

Note: A permutation array contains numbers from 0 to `n-1` exactly once. This guarantees that `nums[i]` will always be a valid index.

2. VISUALIZE THE MAPPING

Given `nums = [0, 2, 1, 5, 3, 4]`



Finding `ans[1]`:

`nums[1]` is 2. → Look at index 2. → Value is 1.

So, `ans[1] = 1`

3. JAVA SOLUTION

```
class Solution {
    public int[] buildArray(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];

        for (int i = 0; i < n; i++) {
            // Read the value at index i, use it as the new index
            ans[i] = nums[nums[i]];
        }

        return ans;
    }
}
```

4. DRY RUN

Input: `nums = [0, 2, 1, 5, 3, 4]`

i	nums[i] (The Target Index)	nums[nums[i]] (The Value)	ans[i] Assigned
0	nums[0] = 0	nums[0] = 0	ans[0] = 0
1	nums[1] = 2	nums[2] = 1	ans[1] = 1
2	nums[2] = 1	nums[1] = 2	ans[2] = 2
3	nums[3] = 5	nums[5] = 4	ans[3] = 4
4	nums[4] = 3	nums[3] = 5	ans[4] = 5
5	nums[5] = 4	nums[4] = 3	ans[5] = 3

Final Output: `[0, 1, 2, 4, 5, 3]`

5. COMPLEXITY

Time: $O(N)$

Space: $O(N)$

We traverse the array once. Looking up `nums[nums[i]]` is an $O(1)$ direct access operation via index. Space is $O(N)$ to store the newly built array.

Problem 1929 — Concatenation of Array

EASY

PATTERN: ARRAY COPY

1. UNDERSTAND THE PROBLEM

Given: An integer array `nums` of length `n`.

Find: An array `ans` of length `2n` where `ans` is formed by appending `nums` to itself.

2. KEY IDEA

THE OFFSET PATTERN

If you are at index `i` in the original array, it maps to two places in the new array:

1. `ans[i]` (The first half)
2. `ans[i + n]` (The second half)

3. JAVA SOLUTION

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] ans = new int[2 * n];

        for (int i = 0; i < n; i++) {
            ans[i] = nums[i];           // Fill first half
            ans[i + n] = nums[i];       // Fill second half
        }

        return ans;
    }
}
```

4. COMPLEXITY

Time: $O(N)$

Space: $O(N)$

Even though the output array is $2N$, dropping the constant gives $O(N)$ time and space complexity.

Problem 1470 — Shuffle the Array

EASY

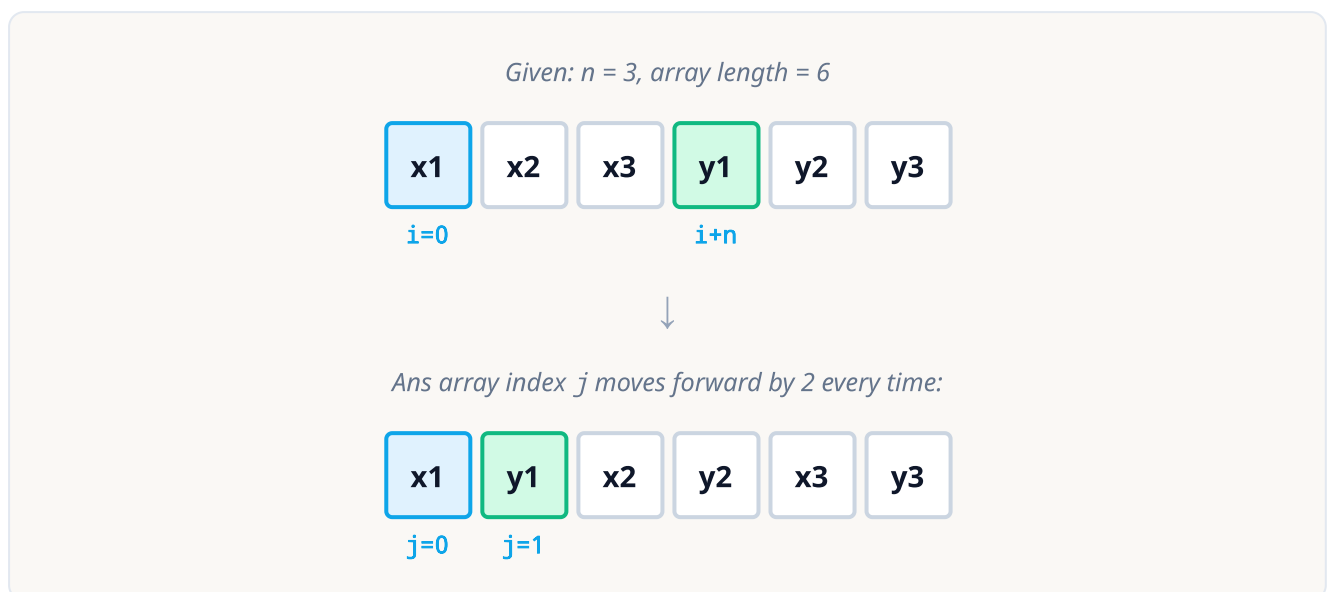
PATTERN: TWO POINTERS

1. UNDERSTAND THE PROBLEM

Given: An array `nums` consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$, and the integer n .

Find: Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

2. VISUALIZE THE INTERLEAVING



3. KEY IDEA

Use a read pointer i that goes from 0 to $n-1$. In each step, we read two elements:

- The "x" element is at `nums[i]`.
- The "y" element is at `nums[i + n]`.

Use a separate write pointer j to place these two elements consecutively into the answer array.

4. JAVA SOLUTION

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[2 * n];
        int j = 0; // Write pointer for ans array

        for (int i = 0; i < n; i++) {
            ans[j++] = nums[i];           // Write x, then increment j
            ans[j++] = nums[i + n];      // Write y, then increment j
        }

        return ans;
    }
}
```

5. DRY RUN

Input: nums = [2, 5, 1, 3, 4, 7], n = 3

i	nums[i] (x)	nums[i+n] (y)	j before	Write Actions	j after
0	2	3	0	ans[0]=2, ans[1]=3	2
1	5	4	2	ans[2]=5, ans[3]=4	4
2	1	7	4	ans[4]=1, ans[5]=7	6

6. COMPLEXITY

Time: $O(N)$

Space: $O(N)$

We loop n times, performing two $O(1)$ insertions per loop. Note that here N refers to the length of the input array (which is $2n$).

Problem 1431 — Kids With Candies

EASY

PATTERN: ARRAY MAX

1. UNDERSTAND THE PROBLEM

Given: An array `candies` where `candies[i]` is the number of candies the *i*-th kid has, and an integer `extraCandies`.

Find: A boolean array (or List) where `result[i]` is true if, after giving the *i*-th kid all the `extraCandies`, they will have the greatest (or equal to greatest) number of candies among all kids.

2. HOW TO THINK ABOUT IT

1. We cannot know if a kid will have the most candies unless we know what the *current maximum* is.
2. **First Pass:** Loop through the array to find the maximum number of candies any kid currently has.
3. **Second Pass:** Loop through again. Check if `kid's candies + extraCandies >= current_maximum`.

3. JAVA SOLUTION

```
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        // Step 1: Find the maximum candies
        int maxCandies = 0;
        for (int candy : candies) {
            maxCandies = Math.max(maxCandies, candy);
        }

        // Step 2: Compare each kid's potential total against the max
        List<Boolean> result = new ArrayList<>();
        for (int candy : candies) {
            if (candy + extraCandies >= maxCandies) {
                result.add(true);
            } else {
                result.add(false);
            }
        }

        return result;
    }
}
```

4. DRY RUN

Input: candies = [2, 3, 5, 1, 3], extraCandies = 3

Pass 1 (Find Max): Max is clearly 5.

Pass 2 (Check condition): candy + 3 >= 5

Kid's Candies	+ Extra (3)	Total	>= Max (5)?	Result Added
2	2 + 3	5	5 >= 5 (Yes)	true
3	3 + 3	6	6 >= 5 (Yes)	true
5	5 + 3	8	8 >= 5 (Yes)	true
1	1 + 3	4	4 >= 5 (No)	false
3	3 + 3	6	6 >= 5 (Yes)	true

5. COMPLEXITY

Time: O(N)

Space: O(N) (for output List)

We loop through the array exactly twice. O(2N) simplifies to O(N).

Problem 1672 — Richest Customer Wealth

EASY

PATTERN: 2D ARRAY

1. UNDERSTAND THE PROBLEM

Given: An $m \times n$ integer grid `accounts` where `accounts[i][j]` is the money the i -th customer has in the j -th bank.

Find: The wealth of the richest customer. (A customer's wealth is the sum of money across all their bank accounts).

2. VISUALIZE THE 2D ARRAY

`accounts = [[1, 2, 3], [3, 2, 1]]`

Customer	Bank 0	Bank 1	Bank 2	Total Wealth
0 (Row 0)	1	2	3	6
1 (Row 1)	3	2	1	6

Max wealth is 6.

3. STEP-BY-STEP APPROACH

1. Initialize `maxWealth = 0`.
2. Use an outer loop to iterate through each customer (each row).
3. For each customer, initialize `currentWealth = 0`.
4. Use an inner loop to iterate through their banks (columns) and add to `currentWealth`.
5. After finishing a customer, update `maxWealth = Math.max(maxWealth, currentWealth)`.
6. Return `maxWealth`.

4. JAVA SOLUTION

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;

        // Loop over each row (customer)
        for (int i = 0; i < accounts.length; i++) {
            int currentWealth = 0;

            // Loop over each column (bank account) for this customer
            for (int j = 0; j < accounts[i].length; j++) {
                currentWealth += accounts[i][j];
            }

            // Update maxWealth if this customer is richer
            maxWealth = Math.max(maxWealth, currentWealth);
        }

        return maxWealth;
    }
}
```

Syntax Tip: You can also use enhanced for-loops here for cleaner code!

```
for (int[] customer : accounts) { ... for (int bank : customer) { ... } }
```

5. COMPLEXITY

Time: $O(M * N)$

Space: $O(1)$

Time: We must visit every single element in the 2D matrix. If there are M customers and N banks, the total operations are

$M * N$.

Space: $O(1)$ because we only keep track of two integer variables (`maxWealth` and `currentWealth`).

Problem 1512 — Number of Good Pairs

EASY

PATTERN: FREQUENCY ARRAY / MATH

1. UNDERSTAND THE PROBLEM

Given: An array of integers `nums`.

Find: The number of "good pairs". A pair (i, j) is called good if `nums[i] == nums[j]` and $i < j$.

2. JAVA SOLUTION (OPTIMIZED $O(N)$)

```
class Solution {
    public int numIdenticalPairs(int[] nums) {
        int[] count = new int[101]; // Max number is 100
        int goodPairs = 0;
        for (int n : nums) { // Add previously seen occurrences to the result
            goodPairs += count[n]; // Register seeing this number
            count[n]++;
        }
        return goodPairs;
    }
}
```

3. DRY RUN

Input: `nums = [1, 2, 3, 1, 1]`

n	goodPairs += count[n]	count[n]++ (Update)	Total Pairs
1	+ 0 (seen zero 1s)	count[1] becomes 1	0
2	+ 0 (seen zero 2s)	count[2] becomes 1	0
3	+ 0 (seen zero 3s)	count[3] becomes 1	0
1	+ 1 (seen one 1 before)	count[1] becomes 2	1
1	+ 2 (seen two 1s before)	count[1] becomes 3	3

4. COMPLEXITY

Time: $O(N)$

Space: $O(1)$

Space is $O(1)$ because the frequency array size is fixed at 101, regardless of how large N gets.

Problem 1365 — How Many Numbers Are Smaller Than the Current Number

EASY

PATTERN: NESTED LOOPS

1. UNDERSTAND THE PROBLEM

Given: An array `nums`.

Find: An array `ans` where `ans[i]` is the count of valid `j` such that `j != i` and `nums[j] < nums[i]`.

2. APPROACH SELECTION

Because the maximum array size is very small ($N = 500$), a brute-force approach utilizing nested loops ($O(N^2)$) is highly readable, robust, and performs perfectly fine. We will compare every number against every other number.

3. JAVA SOLUTION

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];

        // For each number...
        for (int i = 0; i < n; i++) {
            int count = 0;

            // ...compare it to every other number
            for (int j = 0; j < n; j++) {
                // If j is not the same index AND the value is smaller
                if (j != i && nums[j] < nums[i]) {
                    count++;
                }
            }

            ans[i] = count;
        }

        return ans;
    }
}
```

4. DRY RUN

Input: nums = [8, 1, 2]

Outer Loop (i)	Current Num	Inner Loop (j comparisons)	Count	ans[i]
0	8	8 vs 1 (>), 8 vs 2 (>)	2	ans[0] = 2
1	1	1 vs 8 (<), 1 vs 2 (<)	0	ans[1] = 0
2	2	2 vs 8 (<), 2 vs 1 (>)	1	ans[2] = 1

Result: [2, 0, 1]

5. COMPLEXITY

Time: $O(N^2)$

Space: $O(N)$

Time: We loop N times, and inside that loop we loop N times again ($N * N$). While an $O(N)$ solution exists using prefix sum arrays (because numbers are constrained to 0-100), the $O(N^2)$ solution is perfectly acceptable for beginner array foundations.

Problem 1295 — Find Numbers with Even Number of Digits

EASY

PATTERN: DIGIT EXTRACTION

1. UNDERSTAND THE PROBLEM

Given: An array of integers `nums`.

Find: The count of numbers in the array that contain an *even number* of digits.

2. KEY IDEA

COUNTING DIGITS

To count how many digits an integer has, repeatedly divide it by 10 until it reaches 0, keeping track of how many times you divided.

3. JAVA SOLUTION

```
class Solution {
    public int findNumbers(int[] nums) {
        int evenCount = 0;

        for (int num : nums) {
            int digits = 0;
            int temp = num;
            // Count digits mathematically
            while (temp > 0) {
                temp /= 10;
                digits++;
            }
            // Check if the digit count is even
            if (digits % 2 == 0) {
                evenCount++;
            }
        }
        return evenCount;
    }
}
```

4. COMPLEXITY

Time: $O(N * \log_{10}(M))$

Space: $O(1)$

Time: We loop through N numbers. For each number, dividing by 10 takes steps proportional to the

Problem 485 — Max Consecutive Ones

EASY

PATTERN: COUNTING STATE

1. UNDERSTAND THE PROBLEM

Given: A binary array `nums` (containing only 0s and 1s).

Find: The maximum number of consecutive 1s in the array.

2. KEY IDEA

THE BROKEN STREAK

Keep a running counter. Every time you see a 1, increase the counter. Every time you see a 0, the streak is broken, so reset the counter to 0. Update a "max" variable on every step to ensure you don't lose the highest streak.

3. JAVA SOLUTION

```
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int maxOnes = 0;
        int currentOnes = 0;

        for (int num : nums) {
            if (num == 1) {
                currentOnes++;
                // Update maxOnes immediately in case array ends on a 1
                maxOnes = Math.max(maxOnes, currentOnes);
            } else {
                // Streak broken! Reset the current counter
                currentOnes = 0;
            }
        }

        return maxOnes;
    }
}
```

4. DRY RUN

Input: `nums = [1, 1, 0, 1, 1, 1]`

num	Condition	currentOnes	maxOnes
1	<code>num == 1</code>	1	1
1	<code>num == 1</code>	2	2 (New Record!)
0	<code>num != 1</code>	0 (Reset!)	2 (Safely stored)
1	<code>num == 1</code>	1	2
1	<code>num == 1</code>	2	2
1	<code>num == 1</code>	3	3 (New Record!)

Result: 3

Common Mistake: Forgetting to check the maximum if the array ends with 1. Updating `maxOnes` inside the `if (num == 1)` block guarantees we never miss a record!

5. COMPLEXITY

Time: $O(N)$

Space: $O(1)$

We perform exactly one pass through the array, using only two integer variables for state.

Problem 66 — Plus One

EASY

PATTERN: ARRAY MATH

1. UNDERSTAND THE PROBLEM

Given: A large integer represented as an array of digits. The most significant digit is at the head of the array.

Find: The array after adding 1 to the integer.

2. HOW TO THINK ABOUT IT (THE "CARRY")

Adding 1 is easy unless you hit a 9. When you add 1 to 9, it becomes 10, which means the current digit becomes 0 and you carry a 1 over to the left.

- **No Carry:** [1, 2, 3] → Add 1 to last digit → [1, 2, 4] (Done!)
- **Carry Drop:** [1, 2, 9] → 9 becomes 0, loop to left → Add 1 to 2 → [1, 3, 0] (Done!)
- **All Nines:** [9, 9] → Both become 0. We need a new array [1, 0, 0].

3. JAVA SOLUTION

```
class Solution {
    public int[] plusOne(int[] digits) {
        int n = digits.length;

        // Start from the rightmost digit
        for (int i = n - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits; // No carry, return immediately!
            }
            // If it is 9, it becomes 0, and loop continues to carry the 1
            digits[i] = 0;
        }

        // If we finish the loop, it means ALL digits were 9!
        // We need a new array with an extra spot (e.g. 99 -> 100)
        int[] newNumber = new int[n + 1];
        newNumber[0] = 1;

        return newNumber;
    }
}
```

4. DRY RUN (THE 99 CASE)

Input: digits = [9, 9]

i	digits[i]	Condition (< 9)	Action	digits state
1	9	False	digits[1] = 0	[9, 0]
0	9	False	digits[0] = 0	[0, 0]

Loop ends. We create newNumber of size 3 ([0, 0, 0]). We set newNumber[0] = 1. Return [1, 0, 0].

5. COMPLEXITY

Time: $O(N)$

Space: $O(1)$ or $O(N)$

In the worst case (all 9s), we traverse all N digits and create a new array of size $N+1$. For all other cases, space is $O(1)$ as we modify in-place.

The "Read / Write" Pointer Pattern

The final 4 problems in this workbook use the **exact same algorithm pattern.**

Instead of creating a new array, you overwrite the array from the beginning.

1. **Write Pointer (k):** Points to where the *next valid item* should go.
2. **Read Pointer (i):** Scans through the array looking for valid items.
3. If the Read Pointer finds a valid item, it gives it to the Write Pointer, and the Write Pointer steps forward.

Mastering this will permanently level up your array skills.

Problem 27 — Remove Element

EASY

PATTERN: TWO POINTERS (READ/WRITE)

1. UNDERSTAND THE PROBLEM

Given: An array `nums` and an integer `val`.

Find: Remove all occurrences of `val` *in-place*. Return the new length `k`.

2. VISUALIZE THE POINTERS

`nums = [3, 2, 2, 3], val = 3`



`i=1`

`k=0`

i is at 2. Since $2 \neq 3$, we write it at index `k`, then increment `k`.

3. JAVA SOLUTION

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int k = 0; // Write pointer

        // Read pointer (i) scans the whole array
        for (int i = 0; i < nums.length; i++) {
            // If it's a valid number (NOT the value to remove)
            if (nums[i] != val) {
                nums[k] = nums[i]; // Write it
                k++;               // Move write pointer forward
            }
        }
        return k;
    }
}
```

4. COMPLEXITY

Time: $O(N)$

Space: $O(1)$

Problem 26 — Remove Duplicates from Sorted Array

EASY

PATTERN: TWO POINTERS (READ/WRITE)

1. UNDERSTAND THE PROBLEM

Given: An integer array `nums` sorted in non-decreasing order.

Find: Remove duplicates *in-place* so each unique element appears only once. Return the new length `k`.

2. KEY IDEA

THE "NOT EQUAL TO PREVIOUS" RULE

Since the array is sorted, duplicates are always next to each other! The first item is always unique. We start our Write Pointer `k` at index 1. We scan with `i` from index 1. If `nums[i] != nums[i-1]`, it's a brand new number! Write it down.

3. JAVA SOLUTION

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;

        int k = 1; // The first element is always unique

        for (int i = 1; i < nums.length; i++) {
            // If current element is different from the previous one
            if (nums[i] != nums[i - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
}
```

4. COMPLEXITY

Time: $O(N)$

Space: $O(1)$

Problem 283 — Move Zeroes

EASY

PATTERN: TWO POINTERS (READ/WRITE SWAP)

1. UNDERSTAND THE PROBLEM

Given: An integer array `nums`.

Find: Move all 0s to the end of it while maintaining the relative order of the non-zero elements.

2. KEY IDEA

WRITE VALID, FILL REST

Use the exact same Read/Write logic to pack all the non-zero numbers to the front of the array. Once you finish reading the array, you know that from index `k` to the end, everything should just be zeroes!

3. JAVA SOLUTION

```
class Solution {
    public void moveZeroes(int[] nums) {
        int k = 0; // Write pointer

        // Pass 1: Pack all non-zero elements to the front
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[k] = nums[i];
                k++;
            }
        }

        // Pass 2: Fill the remaining array with zeroes
        while (k < nums.length) {
            nums[k] = 0;
            k++;
        }
    }
}
```

Alternative Trick: If you *swap* `nums[i]` and `nums[k]` whenever you find a non-zero, you don't even need the second pass! The zeroes push themselves to the back automatically.

4. COMPLEXITY

Problem 905 — Sort Array By Parity

EASY

PATTERN: TWO POINTERS (READ/WRITE SWAP)

1. UNDERSTAND THE PROBLEM

Given: An integer array `nums`.

Find: Move all the even integers at the beginning of the array followed by all the odd integers. Order within evens/odds does not matter.

2. KEY IDEA

THE MASTER SWAP

This is identical to Move Zeroes! Instead of checking `!= 0`, we check if the number is even (`% 2 == 0`). This time we will use the "Swap" variation of the Two Pointer approach to do it in one single pass.

3. JAVA SOLUTION

```
class Solution {
    public int[] sortArrayByParity(int[] nums) {
        int k = 0; // Write pointer for even numbers

        for (int i = 0; i < nums.length; i++) {
            // If we find an even number, swap it to the front!
            if (nums[i] % 2 == 0) {
                int temp = nums[k];
                nums[k] = nums[i];
                nums[i] = temp;

                k++; // Move write boundary up
            }
        }

        return nums;
    }
}
```

4. COMPLEXITY

Time: $O(N)$

Space: $O(1)$

15-Problem Quick Revision

#	Problem	Core Pattern	Time	Key Takeaway
1480	Running Sum	Prefix Sum	O(N)	<code>ans[i] = ans[i-1] + nums[i]</code>
1920	Build Array from Perm	Array Copy	O(N)	<code>nums[nums[i]]</code> direct access
1929	Concatenation of Array	Array Copy	O(N)	Offset mapping <code>i</code> and <code>i+n</code>
1470	Shuffle the Array	Two Pointers	O(N)	Read <code>x</code> , Read <code>y</code> , write both
1431	Kids With Candies	Array Max	O(N)	Two passes: Find Max, then Add
1672	Customer Wealth	2D Array	O(M*N)	Nested loops for row/col
1512	Number of Good Pairs	Freq Array	O(N)	Store counts in size 101 array
1365	Smaller Than Current	Nested Loops	O(N ²)	Brute force compare <code>i</code> vs <code>j</code>
1295	Even Number of Digits	Digit Math	O(N)	<code>while(n>0) n/=10</code>
485	Max Consecutive Ones	State Count	O(N)	Reset count on 0, track max
66	Plus One	Array Math	O(N)	Carry over 9s. Handle [9,9]
27	Remove Element	Read / Write	O(N)	Write if <code>!= val</code>
26	Remove Duplicates	Read / Write	O(N)	Write if <code>!= prev</code>
283	Move Zeroes	Read / Write	O(N)	Pack non-zeros, fill rest
905	Sort Array By Parity	R/W Swap	O(N)	Swap evens to the <code>k</code> pointer

PATTERN REVISION

The Read / Write Two Pointer

Problems: Remove Element, Remove Duplicates, Move Zeroes, Sort by Parity.

Key Idea: A fast scanner (`i`) finds valid items and feeds them to a slow writer (`k`). This modifies arrays in-place in O(1) space.